

# Compte Rendu — TP7 : Infrastructure as Code (plateforme reproductible)

Mastère Expert IT — CI/CD & DevSecOps

Groupe : gGODON-DAUVEL

Date : 11 juin 2026

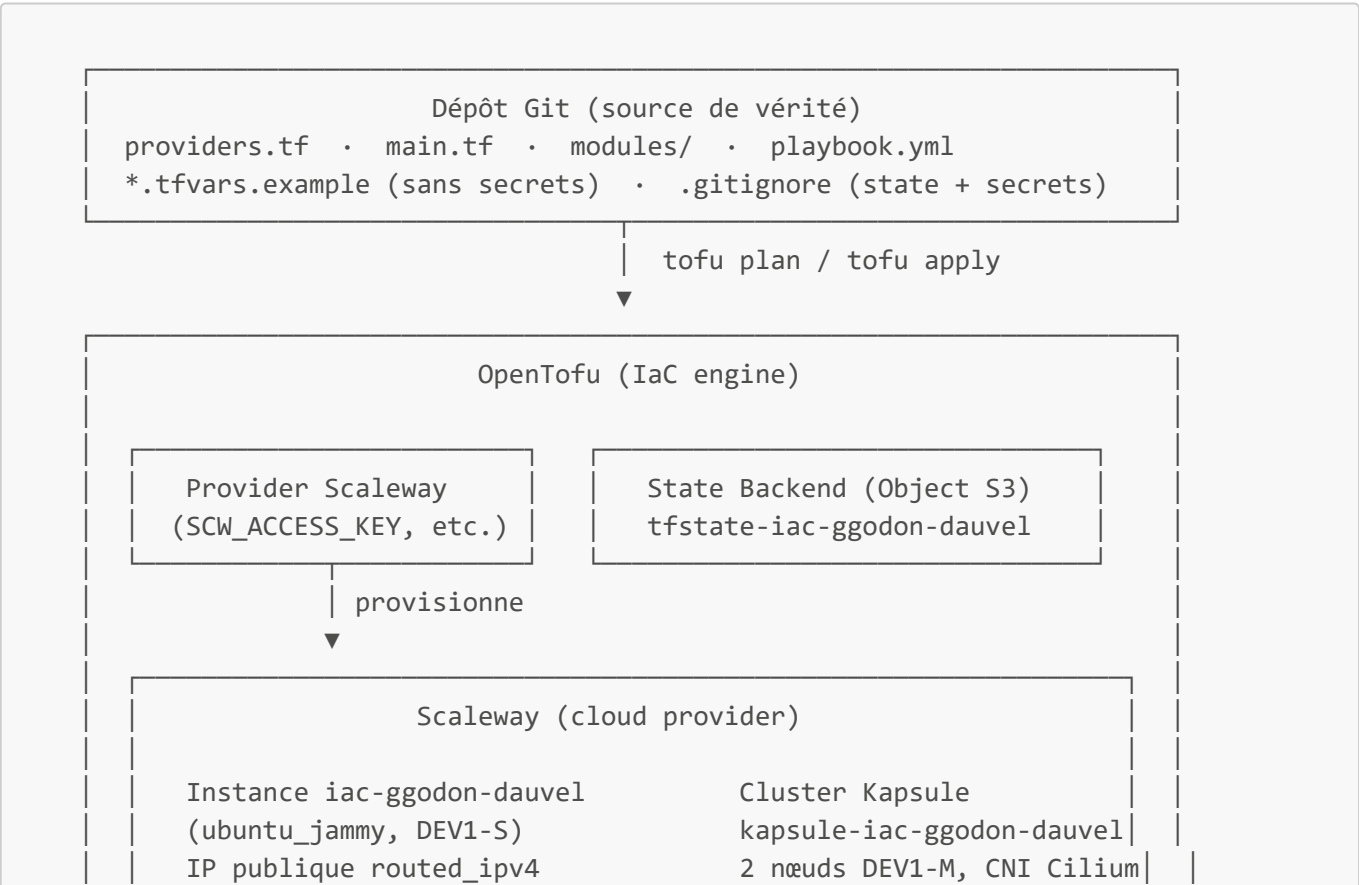
Lien GitHub : [CI-CD\\_SemaineIntensive](#)

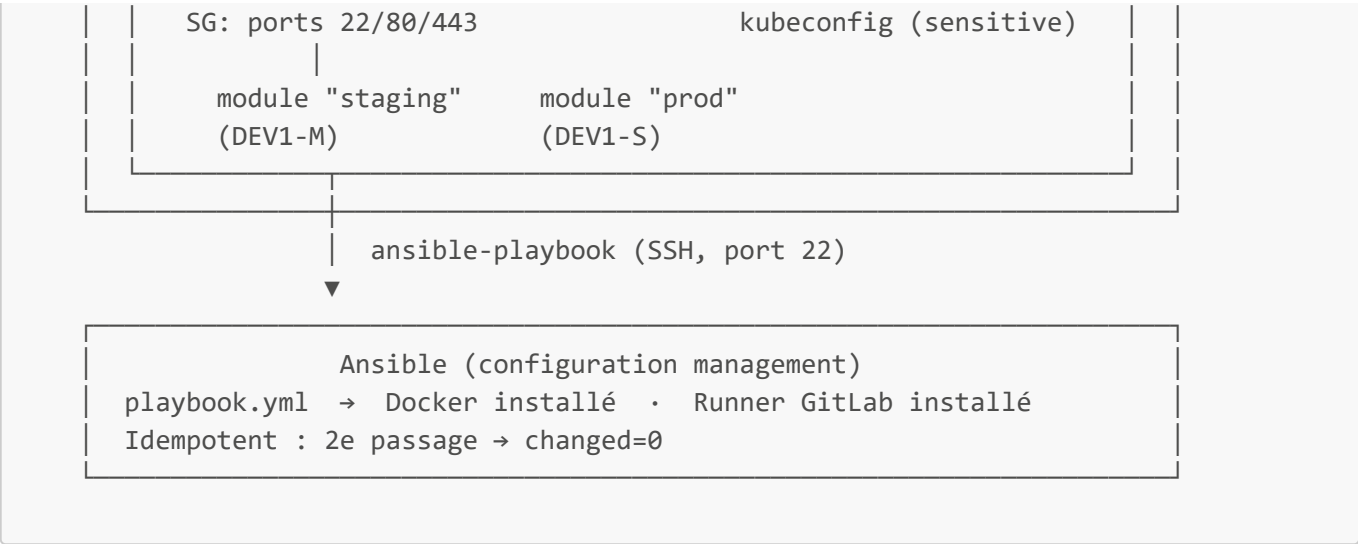
## Table des matières

- 1. [Architecture de l'infrastructure décrite en code](#)
- 2. [Phase 1 — OpenTofu et provider Scaleway](#)
- 3. [Phase 2 — Provisionner une instance en déclaratif](#)
- 4. [Phase 3 — State distant et sécurisé](#)
- 5. [Phase 4 — Modules et variables](#)
- 6. [Phase 5 — Cluster Kapsule en code](#)
- 7. [Phase 6 — Configuration avec Ansible](#)
- 8. [Phase 7 — Destruction et reconstruction](#)
- 9. [Mini-questionnaire](#)

## 1. Architecture de l'infrastructure décrite en code

Schéma de l'infrastructure





Rôle du state et stratégie de stockage retenue

Le **state** OpenTofu/Terraform est un fichier JSON (`terraform.tfstate`) qui représente la correspondance entre les ressources déclarées dans le code et les ressources réelles dans le cloud. Il contient les identifiants Scaleway, les IPs, les métadonnées — éventuellement des secrets en clair. Sans state, OpenTofu ne peut pas calculer le diff (`plan`) entre ce qui existe et ce qui est demandé.

La stratégie retenue est un **backend S3 distant** sur Object Storage Scaleway (`tfstate-iac-ggodon-dauvel`). Ce choix répond à trois contraintes :

| Contrainte             | Backend local                            | Backend S3 distant                    |
|------------------------|------------------------------------------|---------------------------------------|
| Travail en équipe      | Impossible (conflits, désynchronisation) | Un seul state partagé                 |
| Sécurité               | Risque de commit accidentel dans Git     | Hors du dépôt, accès contrôlé par IAM |
| Reprise après incident | State perdu si poste perdu               | Répliqué côté Scaleway                |

Le state n'est **jamais** committé dans Git (`.gitignore : *.tfstate*`).

Structure des modules et paramétrage par environnement

Le module `modules/app-instance/` expose des **variables d'entrée** (`name`, `instance_type`, `ports`) et des **outputs** (`ip_address`, `server_id`). L'instanciation pour staging et prod ne diffère que par les valeurs passées — jamais par une logique interne au module. Ce principe est identique aux `values.yaml` Helm du TP4 et aux overrides Compose du TP2 : un socle commun, des écarts déclarés à la frontière.

Séparation provisioning (OpenTofu) / configuration (Ansible)

| Outil    | Périmètre                                                                                 | Résultat attendu                     |
|----------|-------------------------------------------------------------------------------------------|--------------------------------------|
| OpenTofu | Création et cycle de vie des <b>ressources cloud</b> (instances, réseau, cluster)         | Infrastructure existante, adressable |
| Ansible  | <b>Configuration logicielle</b> sur les machines existantes (paquets, services, fichiers) | Machine dans l'état applicatif voulu |

Les deux phases sont découplées : OpenTofu produit l'IP via un output, Ansible la consomme comme inventaire. Ce couplage minimal permet de remplacer l'un sans toucher à l'autre.

En quoi l'IaC répond au problème des « flocons de neige »

Un serveur « flocon de neige » est une machine qui a divergé de son état d'origine à force de modifications manuelles — SSH, corrections ad hoc, paquets installés à la main. Son comportement est unique, son contenu inconnu, sa reconstruction impossible à reproduire exactement. C'est précisément l'instance GitLab montée manuellement au TP1.

L'IaC répond à ce problème en rendant l'infrastructure **déclarative, versionnée et restructurable** :

- Toute modification passe par un commit Git — l'historique est l'audit trail.
- `tofu apply` converge toujours vers le même état quelle que soit la machine de départ.
- `tofu destroy && tofu apply` recrée l'infrastructure à l'identique en quelques minutes.
- Ansible garantit que la configuration logicielle est idempotente : rejouer le playbook ne brise rien.

---

## 2. Phase 1 — OpenTofu et provider Scaleway

### Capture 1 — `tofu init` réussi avec le provider Scaleway

```
mda@MatthiasD:~/tp7-iac$ tofu init

Initializing the backend...

Initializing provider plugins...
- Finding latest version of scaleway/scaleway...
- Installing scaleway/scaleway v2.76.0...
- Installed scaleway/scaleway v2.76.0 (signed, key ID F5BF26CADF6F9614)

Providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://opentofu.org/docs/cli/plugins/signing/

OpenTofu has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that OpenTofu can guarantee to make the same selections by default when
you run "tofu init" in the future.

OpenTofu has been successfully initialized!

You may now begin working with OpenTofu. Try running "tofu plan" to see
any changes that are required for your infrastructure. All OpenTofu commands
should now work.

If you ever set or change modules or backend configuration for OpenTofu,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Le projet OpenTofu a été initialisé en déclarant le provider Scaleway dans `providers.tf`. Les credentials ne sont jamais écrits dans les fichiers `.tf` : ils sont injectés par variables d'environnement avant chaque session.

```
# providers.tf
terraform {
```

```

    required_providers {
      scaleway = { source = "scaleway/scaleway" }
    }
  }

  provider "scaleway" {
    zone   = "fr-par-1"
    region = "fr-par"
  }

```

```

export SCW_ACCESS_KEY="SCWXXXXXXXXXXXXXXXXX"      # jamais committé
export SCW_SECRET_KEY="xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"
export SCW_DEFAULT_PROJECT_ID="xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"

tofu init

```

`tofu init` télécharge le provider Scaleway depuis le registre public et initialise le répertoire `.terraform/`. Le fichier `.gitignore` a été complété pour exclure ce dossier ainsi que les fichiers d'état :

```

.terraform/
*.tfstate
*.tfstate.backup
*.tfvars

```

**Point de contrôle :** `tofu version` et `tofu init` aboutissent ; le provider Scaleway est téléchargé.

### 3. Phase 2 — Provisionner une instance en déclaratif

#### Capture 2 — `tofu plan (diff)` puis instance créée visible dans la console



L'instance TP1 a été recrée en code déclaratif. Trois ressources sont nécessaires : un groupe de sécurité, une IP publique explicite, et l'instance elle-même.

```

resource "scaleway_instance_security_group" "web" {
  name           = "sg-iac-ggodon-dauvel"
  inbound_default_policy = "drop"

  dynamic "inbound_rule" {
    for_each = [22, 80, 443]
    content {
      action = "accept"
      port   = inbound_rule.value
    }
  }
}

```

```

}

# L'IP publique doit être créée explicitement –
# sans elle l'instance est injoignable (pas de SSH pour Ansible).
resource "scaleway_instance_ip" "public" {
  type = "routed_ipv4"
}

resource "scaleway_instance_server" "app" {
  name          = "iac-ggodon-dauvel"
  type          = "DEV1-S"
  image         = "ubuntu_jammy"
  ip_id         = scaleway_instance_ip.public.id
  security_group_id = scaleway_instance_security_group.web.id
}

output "app_ip" {
  value = scaleway_instance_ip.public.address
}

```

```

tofu plan    # lit le diff : 3 ressources à créer
tofu apply   # provisionne sur Scaleway

```

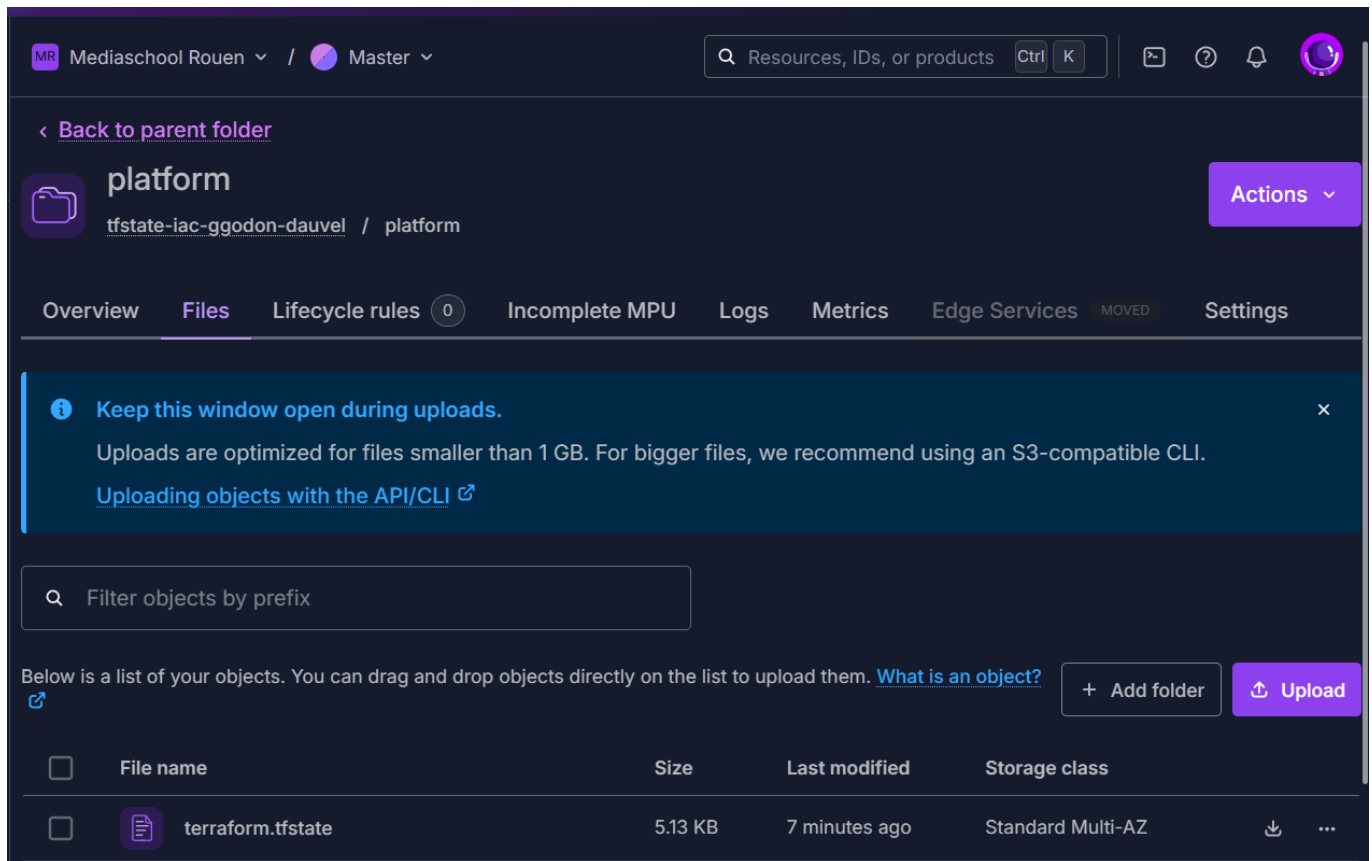
**tofu plan** affiche le diff avant toute modification : chaque ressource à créer est préfixée par **+**. Après **apply**, l'instance **iac-ggodon-dauvel** apparaît dans la console Scaleway avec son IP publique affichée en output.

**Point de contrôle** : L'instance est visible dans la console Scaleway avec une IP publique.

**Piège évité** : Sans la ressource **scaleway\_instance\_ip** et son **ip\_id**, l'instance n'a pas d'IP publique — aucun accès SSH possible, Ansible est bloqué en phase 6. Le state local (**terraform.tfstate**) décrit désormais la réalité : il ne doit jamais être édité à la main.

## 4. Phase 3 — State distant et sécurisé

### Capture 3 — State présent dans le bucket Object Storage



Le state a été migré vers un bucket Object Storage Scaleway (compatible S3). Le bucket a d'abord été créé via la console (ou une ressource `scaleway_object_bucket` appliquée avec le backend encore local), puis le bloc `backend "s3"` a été ajouté à `providers.tf`.

```
terraform {
  required_providers {
    scaleway = { source = "scaleway/scaleway" }
  }

  backend "s3" {
    bucket = "tfstate-iac-ggodon-dauvel"
    key    = "platform/terraform.tfstate"
    region = "fr-par"

    endpoints = { s3 = "https://s3.fr-par.scw.cloud" }

    skip_credentials_validation = true
    skip_region_validation     = true
    skip_requesting_account_id = true
    # skip_s3_checksum = true # à décommenter si erreur de checksum
  }
}
```

```
tofu init -migrate-state # transfère le state local vers le bucket S3
```

**Point de contrôle :** Le fichier `platform/terraform.tfstate` est visible dans le bucket Object Storage ; aucun fichier `.tfstate` n'est présent localement.

### Pièges évités :

- Le bloc `endpoints = { s3 = "..."}`  est la syntaxe requise pour OpenTofu  $\geq 1.6$  ; l'ancien champ `endpoint = "..."` est déprécié et provoque une erreur.
- `skip_requesting_account_id = true` est indispensable pour un S3 non-AWS.
- Le backend S3 Scaleway ne fournit pas de verrouillage natif : une discipline d'équipe (un `apply` à la fois) est nécessaire.

## 5. Phase 4 — Modules et variables

### Capture 4 — Module instancié pour staging et prod, outputs affichés

```
mda@MatthiasD:~/tp7-iac$ tofu init

Initializing the backend...
Initializing modules...

Initializing provider plugins...
- Finding latest version of hashicorp/scaleway...
- Reusing previous version of scaleway/scaleway from the dependency lock file
- Using previously-installed scaleway/scaleway v2.76.0
- Installing hashicorp/scaleway v2.76.0...
- Installed hashicorp/scaleway v2.76.0 (signed, key ID F5BF26CADF6F9614)

Providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://opentofu.org/docs/cli/plugins/signing/

OpenTofu has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

OpenTofu has been successfully initialized!

You may now begin working with OpenTofu. Try running "tofu plan" to see
any changes that are required for your infrastructure. All OpenTofu commands
should now work.

If you ever set or change modules or backend configuration for OpenTofu,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
mda@MatthiasD:~/tp7-iac$ tofu plan
scaleway_instance_ip.public: Refreshing state... [id=fr-par-1/edaf5633-f4f1-435d-b605-04c1544ca875]
scaleway_object_bucket.tfstate: Refreshing state... [id=fr-par/tfstate-iac-ggodon-dauvel]
scaleway_instance_security_group.web: Refreshing state... [id=fr-par-1/de1d3983-023c-4a48-badf-6a6b80374057]
scaleway_instance_server.app: Refreshing state... [id=fr-par-1/7b29521c-d2f1-4e05-92b9-7178c553986b]

OpenTofu used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  + create
  - destroy

OpenTofu will perform the following actions:

# scaleway_instance_ip.public will be destroyed
# (because scaleway_instance_ip.public is not in configuration)
- resource "scaleway_instance_ip" "public" {
  - address      = "51.158.103.127" -> null
  - id           = "fr-par-1/edaf5633-f4f1-435d-b605-04c1544ca875" -> null
  - organization_id = "6c42162c-c73a-4f80-b0fa-4980a486650f" -> null
  - prefix       = "51.158.103.127/32" -> null
  - project_id    = "c03fb7ae-2527-423b-8e6b-3abb5b0ddc34" -> null
  - server_id     = "fr-par-1/7b29521c-d2f1-4e05-92b9-7178c553986b" -> null
```

```

- type = "routed_ipv4" -> null
- zone = "fr-par-1" -> null
}

# scaleway_instance_security_group.web will be destroyed
# (because scaleway_instance_security_group.web is not in configuration)
- resource "scaleway_instance_security_group" "web" {
  - enable_default_security = true -> null
  - external_rules = false -> null
  - id = "fr-par-1/de1d3983-023c-4a48-badf-6a6b80374057" -> null
  - inbound_default_policy = "drop" -> null
  - name = "sg-iac-ggodon-dauvel" -> null
  - organization_id = "6c42162c-c73a-4f80-b0fa-4980a486650f" -> null
  - outbound_default_policy = "accept" -> null
  - project_id = "c03fb7ae-2527-423b-8e6b-3abb5b0ddc34" -> null
  - stateful = true -> null
  - tags = [] -> null
  - zone = "fr-par-1" -> null

  - inbound_rule {
    - action = "accept" -> null
    - port = 22 -> null
    - protocol = "TCP" -> null
  }
  - inbound_rule {
    - action = "accept" -> null
    - port = 80 -> null
    - protocol = "TCP" -> null
  }
  - inbound_rule {
    - action = "accept" -> null
    - port = 443 -> null
    - protocol = "TCP" -> null
  }
}

# scaleway_instance_server.app will be destroyed
# (because scaleway_instance_server.app is not in configuration)
- resource "scaleway_instance_server" "app" {
  - additional_volume_ids = [] -> null
  - boot_type = "local" -> null
  - enable_dynamic_ip = false -> null
  - id = "fr-par-1/7b29521c-d2f1-4e05-92b9-7178c553986b" -> null
  - image = "ubuntu_jammy" -> null
  - ip_id = "fr-par-1/edaf5633-f4f1-435d-b605-04c1544ca875" -> null
  - ip_ids = [] -> null
  - name = "iac-ggodon-dauvel" -> null
  - organization_id = "6c42162c-c73a-4f80-b0fa-4980a486650f" -> null
  - project_id = "c03fb7ae-2527-423b-8e6b-3abb5b0ddc34" -> null
  - protected = false -> null
  - replace_on_type_change = false -> null
  - security_group_id = "fr-par-1/de1d3983-023c-4a48-badf-6a6b80374057" -> null
  - state = "started" -> null
  - type = "DEV1-S" -> null
  - user_data = {
    - "ssh-host-fingerprints" = <<-EOT
      256 SHA256:kUth2BVgsxU5qDEbPKMsweH1+0jAXVhZTzlujdlLEAs root@iac-ggodon-dauvel (ECDSA)
      256 SHA256:lKPgnbaM7KMuAjiBdmon7y6VGMoBnBxRNKwQoYAZcwk root@iac-ggodon-dauvel (ED25519)
      3072 SHA256:gsylLJyTwUjPj0b3rEx6E2ELohJ50Mmb9xw0r6eac2I root@iac-ggodon-dauvel (RSA)
    EOT
  } -> null
  - zone = "fr-par-1" -> null

  - public_ips {
    - address = "51.158.103.127" -> null
    - dynamic = false -> null
    - family = "inet" -> null
    - gateway = "62.210.0.1" -> null
    - id = "fr-par-1/edaf5633-f4f1-435d-b605-04c1544ca875" -> null
    - netmask = "32" -> null
    - provisioning_mode = "dhcp" -> null
  }

  - root_volume {
    - boot = false -> null
    - delete_on_termination = true -> null
    - name = "Ubuntu 22.04 Jammy Jellyfish_sbs_volume_0" -> null
    - sbs_iops = 5000 -> null
    - size_in_gb = 10 -> null
    - volume_id = "fr-par-1/725c3ff6-b549-4402-904e-f55adbce738" -> null
    - volume_type = "sbs_volume" -> null
  }
}

```



```

# scaleway_object_bucket.tfstate will be destroyed
# (because scaleway_object_bucket.tfstate is not in configuration)
- resource "scaleway_object_bucket" "tfstate" {
  - acl                = "private" -> null
  - api_endpoint       = "https://s3.fr-par.scw.cloud" -> null
  - endpoint           = "https://tfstate-iac-ggodon-dauvel.s3.fr-par.scw.cloud" -> null
  - force_destroy      = false -> null
  - id                 = "fr-par/tfstate-iac-ggodon-dauvel" -> null
  - name               = "tfstate-iac-ggodon-dauvel" -> null
  - object_lock_enabled = false -> null
  - project_id         = "c03fb7ae-2527-423b-8e6b-3abb5b0ddc34" -> null
  - region             = "fr-par" -> null
  - tags               = {} -> null

  - versioning {
    - enabled = false -> null
  }
}

# module.prod.scaleway_instance_ip.this will be created
+ resource "scaleway_instance_ip" "this" {
  + address      = (known after apply)
  + id           = (known after apply)
  + organization_id = (known after apply)
  + prefix       = (known after apply)
  + project_id   = (known after apply)
  + reverse      = (known after apply)
  + server_id    = (known after apply)
  + type         = "routed_ipv4"
}

# module.prod.scaleway_instance_security_group.this will be created
+ resource "scaleway_instance_security_group" "this" {
  + enable_default_security = true
  + external_rules          = false
  + id                      = (known after apply)
  + inbound_default_policy  = "drop"
  + name                    = "sg-prod-iac-ggodon-dauvel"
  + organization_id         = (known after apply)
  + outbound_default_policy = "accept"
  + project_id              = (known after apply)
  + stateful                = true

  + inbound_rule {
    + action = "accept"
    + port   = 22
    + protocol = "TCP"
  }
  + inbound_rule {
    + action = "accept"
    + port   = 80
    + protocol = "TCP"
  }
  + inbound_rule {
    + action = "accept"
    + port   = 443
    + protocol = "TCP"
  }
}

# module.prod.scaleway_instance_server.this will be created
+ resource "scaleway_instance_server" "this" {
  + boot_type      = "local"
  + bootscript_id  = (known after apply)
  + cloud_init     = (known after apply)
  + enable_dynamic_ip = false
  + id             = (known after apply)
  + image          = "ubuntu_jammy"
  + ip_id          = (known after apply)
  + name           = "prod-iac-ggodon-dauvel"
  + organization_id = (known after apply)
  + placement_group_policy_respected = (known after apply)
  + project_id     = (known after apply)
  + protected      = false
  + replace_on_type_change = false
  + security_group_id = (known after apply)
  + state          = (known after apply)
  + type           = "DEV1-S"
  + user_data      = (known after apply)

  + filesystems (known after apply)
}

```

```

+ private_ips (known after apply)

+ public_ips (known after apply)

+ root_volume (known after apply)
}

# module.staging.scaleway_instance_ip.this will be created
+ resource "scaleway_instance_ip" "this" {
  + address      = (known after apply)
  + id           = (known after apply)
  + organization_id = (known after apply)
  + prefix       = (known after apply)
  + project_id    = (known after apply)
  + reverse      = (known after apply)
  + server_id     = (known after apply)
  + type         = "routed_ipv4"
}

# module.staging.scaleway_instance_security_group.this will be created
+ resource "scaleway_instance_security_group" "this" {
  + enable_default_security = true
  + external_rules          = false
  + id                      = (known after apply)
  + inbound_default_policy  = "drop"
  + name                    = "sg-staging-iac-ggodon-dauvel"
  + organization_id         = (known after apply)
  + outbound_default_policy = "accept"
  + project_id              = (known after apply)
  + stateful                = true

  + inbound_rule {
    + action = "accept"
    + port   = 22
    + protocol = "TCP"
  }
  + inbound_rule {
    + action = "accept"
    + port   = 80
    + protocol = "TCP"
  }
  + inbound_rule {
    + action = "accept"
    + port   = 443
    + protocol = "TCP"
  }
}

# module.staging.scaleway_instance_server.this will be created
+ resource "scaleway_instance_server" "this" {
  + boot_type      = "local"
  + bootscript_id  = (known after apply)
  + cloud_init     = (known after apply)
  + enable_dynamic_ip = false
  + id             = (known after apply)
  + image          = "ubuntu_jammy"
  + ip_id          = (known after apply)
  + name           = "staging-iac-ggodon-dauvel"
  + organization_id = (known after apply)
  + placement_group_policy_respected = (known after apply)
  + project_id      = (known after apply)
  + protected       = false
  + replace_on_type_change = false
  + security_group_id = (known after apply)
  + state           = (known after apply)
  + type            = "DEV1-S"
  + user_data       = (known after apply)

  + filesystems (known after apply)

  + private_ips (known after apply)

  + public_ips (known after apply)

  + root_volume (known after apply)
}

```

Plan: 6 to add, 0 to change, 4 to destroy.

Changes to Outputs:

```

- app_ip      = "51.158.103.127" -> null
+ prod_ip     = (known after apply)
+ staging_ip   = (known after apply)

```

```
Note: You didn't use the -out option to save this plan, so OpenTofu can't guarantee to take exactly these actions if you run "tofu apply" now.
```

La définition d'instance a été extraite dans un module réutilisable `modules/app-instance/`.

### Structure du module :

```
modules/  
└─ app-instance/  
    │ main.tf      # ressources scaleway_instance_*  
    │ variables.tf # name, instance_type, ports  
    └─ outputs.tf  # ip_address, server_id
```

```
# modules/app-instance/variables.tf  
variable "name" {  
  description = "Nom de l'instance"  
  type        = string  
}  
  
variable "instance_type" {  
  description = "Type d'instance Scaleway"  
  type        = string  
  default     = "DEV1-M"  
}  
  
variable "ports" {  
  description = "Ports ouverts en inbound"  
  type        = list(number)  
  default     = [22, 80, 443]  
}
```

```
# modules/app-instance/outputs.tf  
output "ip" {  
  value = scaleway_instance_ip.public.address  
}  
  
output "id" {  
  value = scaleway_instance_server.app.id  
}
```

### Instanciation pour deux environnements :

```
# main.tf (racine)
module "staging" {
  source      = "./modules/app-instance"
  name        = "staging-iac-ggodon-dauvel"
  instance_type = "DEV1-M"
}

module "prod" {
  source      = "./modules/app-instance"
  name        = "prod-iac-ggodon-dauvel"
  instance_type = "DEV1-S"
}

output "staging_ip" { value = module.staging.ip }
output "prod_ip"    { value = module.prod.ip }
```

**Point de contrôle :** Un même module produit deux environnements distincts ; les outputs IP sont affichés après `tofu apply`.

**Principe clé :** Le module ne contient aucune valeur en dur spécifique à un environnement — tout passe par les variables. C'est le même principe que les `values.yaml` Helm (TP4) et les overrides Compose (TP2) : un socle commun, des écarts déclarés.

## 6. Phase 5 — Cluster Kapsule en code

### Capture 5 — Cluster créé par OpenTofu, `kubectl get nodes` opérationnel



Le cluster Kapsule du TP4 a été reproduit entièrement en code déclaratif. La version Kubernetes est paramétrée par variable pour éviter de figer une version obsolète.

```
# Lister les versions disponibles le jour du TP
scw k8s version list
```

```
variable "k8s_version" {
  description = "Version Kapsule supportée – listez-les avec : scw k8s version list"
  type        = string
}

resource "scaleway_k8s_cluster" "main" {
  name          = "kapsule-iac-ggodon-dauvel"
  version       = var.k8s_version
  cni           = "cilium"
  delete_additional_resources = true
}
```

```
resource "scaleway_k8s_pool" "default" {
  cluster_id = scaleway_k8s_cluster.main.id
  name       = "default"
  node_type  = "DEV1-M"
  size       = 2
}

output "kubeconfig" {
  value     = scaleway_k8s_cluster.main.kubeconfig[0].config_file
  sensitive = true
}
```

```
tofu apply -var="k8s_version=1.35.3"

# Récupérer le kubeconfig et vérifier l'accès
tofu output -raw kubeconfig > kubeconfig.yaml
export KUBECONFIG=kubeconfig.yaml
kubectl get nodes
```

**Point de contrôle :** Le cluster est créé par OpenTofu ; `kubectl get nodes` liste les deux nœuds en état `Ready`.

#### Pièges évités :

- Ne pas figer la version Kubernetes en dur : une version trop ancienne n'est plus proposée par Kapsule et le `apply` échoue — vérifier les versions disponibles le jour du TP.
- Marquer l'output `kubeconfig` comme `sensitive` pour éviter son affichage dans les logs.
- Le CNI Cilium (cohérent avec le TP4) doit être déclaré à la création ; il ne peut pas être modifié après.

## 7. Phase 6 — Configuration avec Ansible

### Capture 6 — Playbook appliqué, puis second run montrant `changed=0`



Ansible configure l'instance provisionnée en phase 2 : Docker et le runner GitLab sont installés de façon idempotente.

```
# playbook.yml
- hosts: app
  become: true
  tasks:
    - name: Installer les dépendances
      apt:
        name:
          - docker.io
```

```

- curl
  update_cache: true

- name: Démarrer Docker
  service:
    name: docker
    state: started
    enabled: true

- name: Installer le runner GitLab
  shell: |
    curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-
runner/script.deb.sh" | bash
    apt-get install -y gitlab-runner
  args:
    creates: /usr/bin/gitlab-runner # idempotence : ne s'exécute que si
absent

```

```

# 1er passage : installation des paquets, changed > 0
ansible-playbook -i "<IP_instance>," -u root playbook.yml

# 2e passage : tout est déjà en place, changed=0
ansible-playbook -i "<IP_instance>," -u root playbook.yml

```

**Point de contrôle :** Le premier passage installe Docker et configure le runner. Le second passage se termine avec `changed=0` — c'est la preuve de l'idempotence.

**Principe d'idempotence :** Les modules Ansible (`apt`, `service`, `copy`, `template`) vérifient l'état actuel avant d'agir. Si l'état souhaité est déjà atteint, rien n'est exécuté. À l'inverse, les tâches `shell` ou `command` s'exécutent à chaque passage sauf si protégées par `creates` ou `when`.

## 8. Phase 7 — Destruction et reconstruction

### Capture 7 — `destroy` puis `apply` : infrastructure reconstruite à l'identique



Cette phase est la démonstration finale de la reproductibilité : l'infrastructure est détruite puis recrée à l'identique depuis le code.

```

# Destruction de toutes les ressources gérées par OpenTofu
tofu destroy

# Vérification : plus aucune ressource visible dans la console Scaleway
# (instances, cluster, IP, security groups)

# Reconstruction à l'identique depuis le code
tofu apply

```

Après **apply**, les ressources recréées sont strictement identiques à celles de la phase 2 : même nom, même type d'instance, mêmes règles de sécurité, même configuration réseau. L'IP publique peut changer (nouvelle allocation), mais la structure est conforme.

**Point de contrôle :** Après **destroy** puis **apply**, l'infrastructure est reconstruite conforme à sa description dans le code.

#### Pièges évités :

- **tofu destroy** détruit réellement : vérifier le périmètre avant d'exécuter (ne pas inclure de ressources partagées avec d'autres TP).
- Les données non gérées par le code (volumes, contenu applicatif dans l'instance) ne survivent pas à **destroy** — c'est l'intérêt de séparer données et infrastructure dans deux stacks distinctes.

## 9. Mini-questionnaire

### 1. Qu'est-ce que le state d'OpenTofu/Terraform et pourquoi ne jamais le committer ?

Le **state** est le fichier (**terraform.tfstate**) qui mappe les ressources déclarées dans le code aux ressources réelles dans le cloud. Il stocke les identifiants, IPs, attributs et dépendances de chaque ressource. Sans state, OpenTofu ne peut pas calculer le diff (**plan**) et ne sait pas quelles ressources existent déjà.

Il ne doit **jamais** être committé pour deux raisons :

1. **Sécurité** : il peut contenir des valeurs sensibles en clair (mots de passe, clés API, tokens) issus des outputs ou des attributs de ressources.
2. **Cohérence** : en équipe, chaque développeur aurait une version du state différente, provoquant des conflits et des doubles créations de ressources. Le backend distant est la solution canonique.

### 2. **plan** et **apply** : à quoi sert l'étape **plan** ?

**tofu plan** calcule et affiche le **diff** entre l'état actuel (tel que décrit dans le state) et l'état souhaité (tel que décrit dans les fichiers **.tf**). Il liste les ressources à créer (+), modifier (~) ou détruire (-), sans effectuer aucune modification réelle.

**plan** est une étape de **validation et de revue** avant action. Elle permet :

- De détecter les effets de bord inattendus (ex. une modification qui force le remplacement d'une ressource).
- D'obtenir une approbation humaine avant d'impacter le cloud.
- De servir de plan d'exécution exact que **apply** peut rejouer de manière déterministe.

Lire le plan avant tout **apply** est une règle d'hygiène fondamentale en IaC.

### 3. Qu'est-ce que l'idempotence et comment Ansible la garantit-il ?

L'**idempotence** signifie qu'appliquer une opération plusieurs fois produit exactement le même résultat que l'appliquer une fois. En configuration, cela signifie : rejouer un playbook sur une machine déjà configurée ne modifie rien et ne brise rien.

Ansible garantit l'idempotence via ses **modules** (`apt`, `service`, `copy`, `file`, `template`...) : chaque module commence par lire l'état actuel du système. Si l'état cible est déjà atteint, le module rapporte `ok` et n'exécute rien. Seules les tâches `shell` : ou `command` : brutes ne sont pas idempotentes par défaut — elles doivent être protégées par `creates` : , `when` : ou `changed_when` : `false`.

L'idempotence est la propriété clé qui permet d'utiliser Ansible comme outil de remédiation : on peut rejouer le playbook à tout moment pour s'assurer qu'une machine est dans l'état attendu.

4. Provisioning (OpenTofu) et configuration (Ansible) : quelle différence et pourquoi les séparer ?

| Dimension    | Provisioning (OpenTofu)                                                    | Configuration (Ansible)                                           |
|--------------|----------------------------------------------------------------------------|-------------------------------------------------------------------|
| Quoi         | Créer et gérer les ressources cloud (instances, réseaux, cluster, buckets) | Installer et paramétrer les logiciels sur les machines existantes |
| Cycle de vie | Lié à l'existence de la ressource ( <code>create/destroy</code> )          | Lié à l'état logiciel (peut être rejoué à tout moment)            |
| Interface    | API cloud (Scaleway, AWS...)                                               | SSH sur les machines                                              |
| Résultat     | Infrastructure adressable avec une IP/endpoint                             | Machine dans l'état applicatif voulu                              |

Les séparer apporte trois bénéfices :

- 1. **Clarté des responsabilités** : OpenTofu ne sait pas ce qui tourne dans la machine ; Ansible ne sait pas comment la machine a été créée.
- 2. **Découplage** : on peut reconfigurer une machine sans la recréer, ou recréer l'infrastructure en réappliquant ensuite le même playbook.
- 3. **Testabilité** : on peut tester le playbook Ansible sur n'importe quelle machine Ubuntu, indépendamment de Scaleway.

5. Pourquoi un backend distant pour le state plutôt qu'un fichier local ?

Un state local est problématique à trois niveaux :

| Problème          | State local                                                                   | Backend S3 distant                                    |
|-------------------|-------------------------------------------------------------------------------|-------------------------------------------------------|
| Travail en équipe | Chaque développeur a sa propre version → conflits, doubles créations          | Un seul state partagé, toujours à jour                |
| Sécurité          | Risque de commit accidentel dans Git (clés, IPs, mots de passe)               | Stocké hors du dépôt, accès contrôlé par IAM Scaleway |
| Résilience        | Perte du state si poste perdu → impossible de gérer les ressources existantes | Répliqué et versionné côté Scaleway                   |



Le backend S3 distant (Object Storage Scaleway) est la solution canonique en équipe : il centralise le state, l'isole du dépôt Git et le rend accessible à tous les membres autorisés — ou à la pipeline CI/CD lors d'un **apply** automatisé.

---

## 6. En quoi l'infrastructure as code répond-elle au problème des serveurs « flocons de neige » ?

Un serveur **flocon de neige** est une machine qui a divergé de son état initial à force de modifications manuelles : paquets installés en SSH, configurations corrigées à la main, services démarrés sans documentation. Sa configuration réelle n'est connue que de la personne qui l'a créée. Elle est impossible à reproduire exactement — c'est l'instance GitLab du TP1.

L'IaC résout ce problème sur trois axes :

1. **Déclaratif** : l'état souhaité est décrit dans des fichiers versionnés dans Git. Il n'existe qu'une seule source de vérité.
2. **Idempotent** : **tofu apply** et **ansible-playbook** convergent toujours vers le même état, quelle que soit la machine de départ. Aucune dérive n'est possible si tout passe par le code.
3. **Reconstructible** : **tofu destroy && tofu apply** + rejeu du playbook Ansible recrée l'infrastructure à l'identique en quelques minutes. Le TP7 en fait la démonstration par l'acte (phase 7).

La phrase clé du cours se vérifie ici : **une infrastructure relue, versionnée et reconstructible n'est plus un flocon de neige.**

---

## 7. Terraform et OpenTofu : qu'est-ce qui a motivé le fork ?

En août 2023, HashiCorp a changé la licence de Terraform de la **Mozilla Public License 2.0 (MPL 2.0)** — une licence open source — vers la **Business Source License 1.1 (BSL)**, qui restreint l'utilisation commerciale de Terraform par des concurrents de HashiCorp.

Cette décision a provoqué la création d'**OpenTofu**, fork communautaire maintenu par la **Linux Foundation** sous licence MPL 2.0. Les motivations principales du fork :

- **Neutralité** : OpenTofu est gouverné par la communauté, sans dépendance à un éditeur propriétaire.
- **Compatibilité ascendante** : OpenTofu est un remplacement drop-in de Terraform — mêmes fichiers **.tf**, même syntaxe HCL, même CLI (**tofu** en lieu et place de **terraform**).
- **Pérennité** : les entreprises utilisant Terraform dans des produits commerciaux risquaient des poursuites sous BSL ; OpenTofu est sous licence libre sans ambiguïté.

OpenTofu a depuis publié ses propres fonctionnalités (state encryption, nouvelles fonctions HCL) et est recommandé dans le contexte de ce TP comme alternative entièrement libre.